Asset Allocation EUR in R

A guide to the project: theory, methodology and code

October 2026

Abstract

This document describes the project `portfolio-optimization-r` from beginning to end. Part I states the problem and the method: four euro-denominated asset classes, a 60-business-day horizon, a joint distribution of returns obtained by bootstrapping whole historical trading days, risk measured by the Expected Shortfall at the 95% level, and the minimum-ES portfolio, the maximum return-to-ES portfolio and the mean–ES frontier solved exactly as linear programs in the Rockafellar–Uryasev formulation. Part II walks through the repository folder by folder and function by function, so that a reader can run the notebook, change its parameters, extend the universe and understand what every test checks. The project is didactic: the code favours explicit steps over clever constructs, and this document is meant to be read next to it.

Contents

1 Introduction **3**

I Theory and methodology **3**

2 The allocation problem **3**
2.1 Setting and notation 3
2.2 What is being optimised 4

3 Data **4**
3.1 Universe 4
3.2 From prices to daily log returns 4
3.3 Cache 5

4 Simulation of returns at the horizon **5**
4.1 The model 5
4.2 Scale and reproducibility 6

5 Risk measure: Expected Shortfall **6**
5.1 Definition 6
5.2 Why ES rather than variance or VaR 6
5.3 Return-to-risk ratio 6

6 Optimisation **7**
6.1 The Rockafellar–Uryasev linear program 7
6.2 Minimum-ES portfolio and the frontier 7
6.3 Maximum return-to-ES ratio with a single LP 8
6.4 Allocation line 9

7 Random portfolios **9**

8 Outputs and the published run **9**
8.1 The two charts 9
8.2 Results of the run of 2026-10-29 9

9 Limitations and possible extensions **11**

II The code **11**

10 Repository layout **11**
10.1 Packages 12

11 The notebook: `Asset Allocation EUR.Rmd` **12**

12 The `R/` folder, file by file **13**
12.1 `params.R`: parameters, horizon helpers and `check_bounds()` 13
12.2 `data.R`: download, cache, prices and log returns 14
12.3 `simulation.R`: the bootstrap 15
12.4 `metrics.R`: portfolio return, ES and ratio 15
12.5 `optimisation.R`: the linear programs 15
12.6 `cloud.R`: random portfolios 16
12.7 `stats.R`: tables and the allocation line 16
12.8 `plots.R`: the two charts 17
12.9 `all.R` 17

13 Tests and verification **17**
13.1 Unit tests on synthetic data: `tests/testthat/` 17
13.2 Cross-check against a reference implementation: `tests/cross_check/` 18
13.3 Brute-force checks on the real data: `tests/verify_real_data.R` 19

14 Running and modifying the project **19**
14.1 First run 19
14.2 Common changes 19
14.3 Running the tests 20

1 Introduction

The project answers a classical question of portfolio theory with a non-classical risk measure. Given a set of asset classes and their price history, how should an investor who holds the portfolio unchanged for about three months split the money between them? Markowitz's answer measures risk by the variance of returns and leads to the familiar mean–variance frontier. This project keeps the structure of that answer, a frontier of portfolios that trade expected return against risk and one portfolio that makes the best use of risk, but replaces the variance with the Expected Shortfall (ES), the average loss in the worst 5% of outcomes. Two consequences follow. First, the distribution of returns at the horizon must be known in more detail than its mean and covariance: it is obtained by simulation, by resampling the historical days. Second, the optimisation problems are no longer quadratic programs but linear ones, which a modern solver handles exactly and in a fraction of a second.

The repository contains an R Markdown notebook that runs the whole analysis and produces two charts, a folder of R scripts holding the functions the notebook calls, and a test suite. Everything the notebook prints can be reproduced from a fresh clone in two steps, up to the fact that prices are downloaded up to the current day.

This document has two parts. Part I is self-contained and can be read without opening the code; it fixes notation, derives the linear programs and discusses the modelling choices and their limits. Part II assumes Part I and explains the code, file by file. Readers who want to run the notebook first should start from Section 14.

Part I
Theory and methodology

2 The allocation problem

2.1 Setting and notation

The universe consists of $n = 4$ asset classes indexed by $i = 1, \dots, n$. A portfolio is a weight vector $\mathbf{w} \in \mathbb{R}^n$: w_i is the fraction of the money invested in asset i. Two kinds of constraints apply throughout:

- *fully invested*: $\mathbf{1}^\top \mathbf{w} = \sum_i w_i = 1$, no cash left over and no leverage;
- *per-asset bounds*: $\ell_i \le w_i \le u_i$, with $0 \le \ell_i \le u_i \le 1$. The default is $\ell_i = 0$ and $u_i = 1$, which is the long-only case (no short sales).

The set of admissible portfolios, a bounded simplex, is non-empty if and only if $\sum_i \ell_i \le 1 \le \sum_i u_i$; the code checks this before any optimisation.

The horizon is $H = 60$ business days, about three months. The portfolio is *buy-and-hold*: the weights are set at the start of the horizon and not touched until its end. Uncertainty about the future is represented by S equally likely scenarios, $s = 1, \dots, S$, with $S = 10\,000$. In scenario s asset i earns the simple return $R_{s,i}$ over the horizon; the $S \times n$ matrix R of these returns is the *scenario matrix*. The portfolio return in scenario s is then the weighted sum

$$\rho_s(\mathbf{w}) \;=\; \sum_{i=1}^{n} R_{s,i}\, w_i \;=\; R_s \mathbf{w}, \tag{1}$$

where R_s is row s of R. This is exact for a buy-and-hold portfolio measured in simple returns, which is why simple (not logarithmic) returns are used at the horizon.

The expected return of asset i at the horizon is the scenario average $\bar{r}_i = \frac{1}{S}\sum_s R_{s,i}$, and the expected portfolio return is $\mathbb{E}[\rho(\mathbf{w})] = \bar{\mathbf{r}}^\top \mathbf{w}$, linear in the weights.

Finally, $\alpha = 0.95$ is the confidence level of the Expected Shortfall and r_f the annual risk-free rate, converted to the horizon as $r_f^{(H)} = \exp(r_f H/252) - 1$. The published run uses $r_f = 0$, so $r_f^{(H)} = 0$; the formulas keep $r_f^{(H)}$ because the code does.

2.2 What is being optimised

Three objects are computed, each replacing a familiar one from mean–variance analysis:

1. the *minimum-ES portfolio*, the admissible portfolio with the smallest Expected Shortfall (the analogue of the global minimum-variance portfolio);
2. the *mean–ES efficient frontier*, the minimum-ES portfolio for every attainable level of expected return (the analogue of the efficient frontier);
3. the *maximum-ratio portfolio*, the one that maximises expected excess return per unit of Expected Shortfall (the analogue of the tangency, or maximum-Sharpe, portfolio).

A cloud of random admissible portfolios and the single assets are added to the chart to show where the frontier sits relative to everything else that could have been chosen.

3 Data

3.1 Universe

Each asset class is proxied by a UCITS exchange-traded fund quoted on Xetra in euro (Table 1). Prices are the *adjusted* closes published by Yahoo Finance, which fold dividends and other distributions back into the price; each series therefore behaves like the total-return index of its asset class, net of the fund's costs. ETFs rather than the indices themselves are used because free sources do not publish total-return bond indices.

Asset class	Ticker	Fund
Euro government bonds	`EUNH.DE`	iShares Core € Govt Bond
Euro corporate IG	`EUN5.DE`	iShares Core € Corp Bond
Euro corporate HY	`EUNW.DE`	iShares € High Yield Corp Bond
European equity	`EXSA.DE`	iShares STOXX Europe 600

Table 1: Investment universe. IG = investment grade, HY = high yield.

3.2 From prices to daily log returns

Let $P_{t,i}$ be the adjusted close of asset i on trading day t. The daily log return is

$$x_{t,i} \;=\; \log \frac{P_{t,i}}{P_{t-1,i}} \;=\; \log P_{t,i} - \log P_{t-1,i}. \tag{2}$$

Log returns are used at the daily level because they add up over time: the log return over several days is the sum of the daily ones, which is what the simulation of Section 4 needs.

The four series do not share exactly the same calendar. A day on which one exchange is closed and another is open gives a missing price; it is filled with the last available price of that asset (forward fill), so that the asset shows a zero return on that day. Days on which no asset has a price (common holidays) are dropped altogether. Finally the series are aligned on their common dates, so the sample starts when the youngest fund was launched: the high-yield ETF, in December 2010. The run of 2026-10-29 uses prices from 2010-12-15 to 2026-10-27, i.e. $T = 4\,027$ daily returns. The $T \times n$ matrix X of daily log returns is the only input of everything that follows.

Weekly returns. The parameter `RETURN_FREQ` (default `"daily"`) can switch the whole analysis to weekly returns. With `"weekly"` the daily log returns are summed within each ISO calendar week, which gives the log return between the last prices of consecutive weeks; a week shortened by a holiday is still a week, and the first and last weeks of the sample are dropped because they may be partial. Each weekly return is dated at its last trading day. The horizon stays expressed in business days and is converted to a number of weeks (60 days = 12 weeks); the risk-free conversion is unchanged. One caveat: because of holidays a resampled week has on average about 4.9 trading days, so 12 weeks cover about 58 trading days rather than 60, and drift and dispersion at the horizon are correspondingly about 3% lower than in daily mode. The sample of the run of 2026-10-29 (4 027 daily returns) gives 827 weekly returns.

3.3 Cache

Prices are downloaded from 2010-01-01 to the current day and cached on disk, with a 12-hour time-to-live. Two rules protect the cache. The full history is always re-downloaded, never appended: Yahoo restates the whole adjusted series after every distribution, so appending fresh rows to old ones would splice two different adjustment bases and create a spurious jump in the returns. And a partial download (one ticker missing) never overwrites a complete one, so that a transient failure of the data source cannot become permanent. Section 12.2 describes the mechanism.

4 Simulation of returns at the horizon

4.1 The model

The model assumes that the vectors of daily log returns $x_t = (x_{t,1}, \ldots, x_{t,n})$ are independent and identically distributed across days, with the cross-sectional dependence between assets observed in the sample. Nothing is assumed about the shape of the distribution of x_t: it is the empirical distribution of the T historical days.

Under this assumption the horizon return of a scenario is obtained by drawing H days at random, with replacement, from the sample and summing their log returns (a bootstrap in the sense of Efron [5]). With weekly returns the same holds with "week" for "day" and $H = 12$ weeks: the text below says days for brevity. For scenario s draw $\tau_{s,1}, \ldots, \tau_{s,H}$ uniformly from $\{1, \ldots, T\}$ and set

$$C_{s,i} = \sum_{h=1}^{H} x_{\tau_{s,h},\, i}, \qquad R_{s,i} = \exp(C_{s,i}) - 1. \tag{3}$$

$C_{s,i}$ is the horizon log return and $R_{s,i}$ the corresponding simple return, the one that enters (1).

The essential point is that the *same* day index $\tau_{s,h}$ is used for all n assets: a day is resampled as a whole row of X. Every scenario is therefore a sum of actual historical cross-sections, and the correlations between assets, their joint behaviour in crashes, the skewness and the fat tails of each series are reproduced exactly as they occurred, without being modelled. What is *not* reproduced is the dependence between consecutive days: volatility clustering (a turbulent day tends to be followed by another) and autocorrelation are lost, because the days are drawn independently. Section 9 returns to this.

The expected horizon log return of asset i is $H\bar{x}_i$, where $\bar{x}_i$ is the historical mean of its daily log returns. The drift of each asset is thus its historical average, a deliberate choice: the project does not attempt to forecast returns, it asks what allocation the historical distribution would recommend.

4.2 Scale and reproducibility

With $S = 10\,000$ scenarios and $H = 60$ days the simulation draws 600 000 day indices and produces a $10\,000 \times 4$ scenario matrix in well under a second. Two separate random seeds are used, one for the scenarios and one for the random cloud of Section 7, so that changing the size of the cloud does not alter the scenarios. Given the same cached prices and the same seeds, every number in the notebook is reproduced exactly. Because the download runs up to the current day, two runs on different days differ even with identical code.

5 Risk measure: Expected Shortfall

5.1 Definition

Fix a portfolio and let ρ_s be its return in scenario s. The *loss* is $L_s = -\rho_s$. The Expected Shortfall at level α is the average loss over the worst $(1-\alpha)$ share of the scenarios. With S equally likely scenarios the size of that tail is $m = (1-\alpha)S$, which need not be an integer. Let $L_{(1)} \geq L_{(2)} \geq \cdots \geq L_{(S)}$ be the losses in decreasing order and $k = \lfloor m \rfloor$. Then

$$\mathrm{ES}_\alpha \;=\; \frac{1}{m}\Big(\sum_{j=1}^{k} L_{(j)} + (m-k)\, L_{(k+1)}\Big)\,: \tag{4}$$

the k largest losses count in full and the next one with the fractional weight $m-k$. This is the exact conditional value-at-risk of a discrete distribution (the average of the loss quantile function over $[\alpha, 1]$), and it is the quantity that the linear program of Section 6 minimises, for any S and α. With $S = 10\,000$ and $\alpha = 0.95$ one has $m = 500$, an integer (up to floating-point rounding), and (4) is simply the average of the 500 largest losses.

ES is reported as a *positive number*: $\mathrm{ES}_{95\%} = 0.04$ means that, averaging over the worst 5% of scenarios, the portfolio loses 4% of its value over the horizon. The notebook also reports the value-at-risk, as the $(1-\alpha)$ quantile of the *returns*: $\mathrm{VaR}_\alpha = q_{1-\alpha}(\rho)$, a return, usually negative. The two sign conventions differ and both are stated next to the table that uses them.

5.2 Why ES rather than variance or VaR

Variance penalises gains and losses alike and summarises the whole distribution by one number that says nothing about the tail. VaR gives the threshold of the tail but ignores what happens beyond it, and it is not sub-additive: diversifying can increase it. ES is *coherent* in the sense of Artzner et al. [3], in particular sub-additive, and it looks precisely at the part of the distribution an investor fears. Two of its properties are used explicitly in the optimisation:

1. *positive homogeneity*: scaling every scenario return by $t > 0$ scales the ES by t, so $\mathrm{ES}_\alpha(R\,(t\mathbf{w})) = t\ \mathrm{ES}_\alpha(R\mathbf{w})$;
2. *translation*: adding a sure return c to every scenario lowers the ES by c: $\mathrm{ES}_\alpha(\rho + c) = \mathrm{ES}_\alpha(\rho) - c$.

In particular, the ES of the *excess* returns $\rho - r_f^{(H)}$ equals $\mathrm{ES}_\alpha(\rho) + r_f^{(H)}$, and a riskless position earning $r_f^{(H)}$ has $\mathrm{ES}_\alpha = -r_f^{(H)}$: a sure gain is a negative loss.

5.3 Return-to-risk ratio

The analogue of the Sharpe ratio is the expected excess return divided by the ES of the excess returns:

$$\varrho(\mathbf{w}) \;=\; \frac{\mathbb{E}[\rho] - r_f^{(H)}}{\mathrm{ES}_\alpha(\rho - r_f^{(H)})} \;=\; \frac{\bar{\mathbf{r}}^{\top}\mathbf{w} - r_f^{(H)}}{\mathrm{ES}_\alpha(R\mathbf{w}) + r_f^{(H)}}. \tag{5}$$

With $r_f^{(H)} = 0$ this is simply expected return over ES. On fully invested portfolios the numerator equals $\bar{\mathbf{r}}^{e\top}\mathbf{w}$ and the denominator $\mathrm{ES}_\alpha(R^e\mathbf{w})$, with $R^e = R - r_f^{(H)}$ and $\bar{\mathbf{r}}^e = \bar{\mathbf{r}} - r_f^{(H)}\mathbf{1}$ the excess returns; written in this way both are positively homogeneous in $\mathbf{w}$, and that is what the single-LP solution of Section 6.3 relies on. Measuring the denominator on excess returns, rather than on raw returns, is what makes the optimal ratio the slope of the allocation line of Section 6.4.

6 Optimisation

6.1 The Rockafellar–Uryasev linear program

The ES in (4) is defined through a sort, which is not a linear operation. Rockafellar and Uryasev [1, 2] showed that it can be written as the solution of a minimisation problem:

$$\mathrm{ES}_\alpha(\mathbf{w}) \;=\; \min_{\zeta\in\mathbb{R}} \Big\{ \zeta + \frac{1}{(1-\alpha)S} \sum_{s=1}^{S} \max\big(L_s(\mathbf{w}) - \zeta,\, 0\big) \Big\}, \tag{6}$$

where the minimiser $\zeta^\star$ is the value-at-risk of the losses. The intuition: the expression inside the braces is a penalty for choosing a threshold ζ too low (many losses exceed it and the sum grows) or too high (ζ itself grows); the balance is struck exactly at the tail boundary, and the value there is the tail average.

The crucial step is that $L_s(\mathbf{w}) = -R_s\mathbf{w}$ is linear in $\mathbf{w}$. Introduce one auxiliary variable u_s per scenario, meant to stand for $\max(L_s(\mathbf{w}) - \zeta, 0)$. Because the objective pushes every u_s down, it suffices to require $u_s \geq L_s(\mathbf{w}) - \zeta$ and $u_s \geq 0$: at the optimum u_s settles on the larger of the two, which is the positive part. Minimising (6) jointly over $\mathbf{w}$ and ζ then becomes a linear program in the variables $(\mathbf{w}, \zeta, \mathbf{u}) \in \mathbb{R}^n \times \mathbb{R} \times \mathbb{R}^S$:

$$\begin{aligned} \min_{\mathbf{w},\zeta,\mathbf{u}} \quad & \zeta + \frac{1}{(1-\alpha)S} \sum_{s=1}^{S} u_s \\ \text{s.t.} \quad & -R_s\mathbf{w} - \zeta - u_s \;\leq\; 0, \qquad u_s \geq 0, \qquad s = 1, \dots, S, \\ & \mathbf{1}^\top\mathbf{w} = 1, \qquad \ell_i \leq w_i \leq u_i, \\ & \bar{\mathbf{r}}^\top\mathbf{w} \;\geq\; r_{\text{target}} \quad \text{(optional)}. \end{aligned} \tag{7}$$

The program has $n + 1 + S$ variables (10 005 here) and S scenario rows plus a few portfolio rows. The scenario block is the sparse matrix $[\,-R \;\; -\mathbf{1} \;\; -I_S\,]$. The problem is solved with HiGHS [6], an open-source simplex and interior-point solver; one solve with $S = 10\,000$ takes about one second on a laptop, and the answer is exact up to the solver's tolerance (about 10^{-7}). There is no local optimum to worry about: a linear program is convex.

Proposition 1. *The optimal value of* (7) *without the target constraint equals* $\min_{\mathbf{w}} \mathrm{ES}_\alpha(\mathbf{w})$ *over the admissible weights, with* ES_α *as in* (4).

At the optimum u_s equals the positive part of $L_s - \zeta^\star$ and the objective evaluates to (4), fractional boundary weight included. The project keeps a single implementation of (4) and checks, in the tests, that it agrees with the LP optimum to 10^{-8} both when m is an integer and when it is fractional.

6.2 Minimum-ES portfolio and the frontier

The minimum-ES portfolio $\mathbf{w}_{\min}$ solves (7) without the target constraint. The frontier is the set of solutions of (7) with the target constraint for a grid of 50 target returns. The grid runs from the expected return of $\mathbf{w}_{\min}$ (below it the constraint is not binding and the solution would be $\mathbf{w}_{\min}$ again) up to 0.999 times the maximum attainable expected return under the bounds. That

maximum is itself a tiny linear program, $\max \bar{\mathbf{r}}^\top \mathbf{w}$ subject to $\mathbf{1}^\top \mathbf{w} = 1$ and $\ell \le \mathbf{w} \le u$, whose solution concentrates the money on the assets with the highest expected returns as far as the bounds allow; the factor 0.999 keeps the last target strictly feasible. The ES along the frontier is non-decreasing in the target return: asking for more return can never make the minimum achievable tail loss smaller. A target that the solver reports as infeasible is skipped with a message rather than aborting the run.

6.3 Maximum return-to-ES ratio with a single LP

The ratio (5) is not linear, and a direct search over the frontier would only find it approximately. A change of variables due to Charnes and Cooper [4] turns the problem into one linear program, exactly, in two steps.

Step 1: rewrite the ratio with excess returns. Let $R^e = R - r_f^{(H)}$ be the matrix of excess scenario returns and $\bar{\mathbf{r}}^e = \bar{\mathbf{r}} - r_f^{(H)}\mathbf{1}$ the vector of expected excess returns. For a fully invested portfolio, $\mathbf{1}^\top \mathbf{w} = 1$ gives $\bar{\mathbf{r}}^\top \mathbf{w} - r_f^{(H)} = \bar{\mathbf{r}}^{e\top} \mathbf{w}$ and $R\mathbf{w} - r_f^{(H)} = R^e \mathbf{w}$, so

$$\varrho(\mathbf{w}) \;=\; \frac{\bar{\mathbf{r}}^{e\top} \mathbf{w}}{\mathrm{ES}_\alpha(R^e \mathbf{w})}.$$

In this form both numerator and denominator are positively homogeneous of degree one in $\mathbf{w}$: doubling all the weights doubles both and leaves the ratio unchanged. Only the *direction* of $\mathbf{w}$ matters.

Step 2: fix the numerator and drop the budget. Since only the direction matters, the budget constraint can be replaced by any other normalisation. For an admissible $\mathbf{w}$ with $\bar{\mathbf{r}}^{e\top} \mathbf{w} > 0$ set $t = 1/(\bar{\mathbf{r}}^{e\top} \mathbf{w})$ and $\mathbf{y} = t\mathbf{w}$. Then $\bar{\mathbf{r}}^{e\top} \mathbf{y} = 1$ and $\varrho(\mathbf{w}) = 1/\,\mathrm{ES}_\alpha(R^e \mathbf{y})$. Maximising ϱ is therefore the same as *minimising* the ES of $R^e \mathbf{y}$ over $\{\mathbf{y} \ge 0 : \bar{\mathbf{r}}^{e\top} \mathbf{y} = 1\}$, with the budget constraint removed; the weights are recovered at the end as $\mathbf{w}^\star = \mathbf{y}^\star / \mathbf{1}^\top \mathbf{y}^\star$ and the optimal ratio is $1/\,\mathrm{ES}_\alpha(R^e \mathbf{y}^\star)$. The per-asset bounds must be rewritten in terms of $\mathbf{y}$. Since $w_i = y_i / \mathbf{1}^\top \mathbf{y}$ and $\mathbf{1}^\top \mathbf{y} > 0$, $\ell_i \le w_i \le u_i$ is equivalent to

$$\ell_i\, \mathbf{1}^\top \mathbf{y} - y_i \;\le\; 0, \qquad y_i - u_i\, \mathbf{1}^\top \mathbf{y} \;\le\; 0, \tag{8}$$

two linear inequalities per asset. The whole problem is then (7) with $\mathbf{w}$ replaced by $\mathbf{y}$, the scenario matrix R^e, the variable bounds $y_i \ge 0$ (no cap), no budget row, the equality $\bar{\mathbf{r}}^{e\top} \mathbf{y} = 1$ and the $2n$ rows (8).

Two preconditions are checked, one before and one after the solve, each stopping the run with an explicit error:

1. some admissible portfolio must have a positive expected excess return, otherwise the normalisation $\bar{\mathbf{r}}^{e\top} \mathbf{y} = 1$ is infeasible; this is read off the maximum-return LP;
2. the optimal ES of the excess returns must be positive. If some admissible portfolio beats the risk-free rate even on average over its worst scenarios, the ratio is unbounded above, and the LP, which minimises the denominator, would silently return a meaningless point with a negative "ratio". The guard does not fire with the present data; it exists so that a change of data or of risk-free rate cannot pass unnoticed.

Remark 1. With $r_f^{(H)} = 0$ excess and raw returns coincide and Step 1 is empty. When $r_f^{(H)} \neq 0$, Step 1 uses the budget to rewrite numerator and denominator as $\bar{\mathbf{r}}^{e\top} \mathbf{w}$ and $\mathrm{ES}_\alpha(R^e \mathbf{w})$, which are homogeneous in $\mathbf{w}$ without any budget; the original expressions $\bar{\mathbf{r}}^\top \mathbf{w} - r_f^{(H)}$ and $\mathrm{ES}_\alpha(R\mathbf{w}) + r_f^{(H)}$ are not, because of their constant terms. Once the budget is dropped in Step 2 only the homogeneous forms may be used.

6.4 Allocation line

Mixing a fraction $t \in [0,1]$ of the maximum-ratio portfolio P with a fraction $1-t$ of the riskless asset gives, by homogeneity and translation, expected return $y = r_f^{(H)} + t\,(\mathbb{E}[\rho_P] - r_f^{(H)})$ and Expected Shortfall of raw returns $x = t\ \mathrm{ES}_\alpha(\rho_P) - (1-t)\,r_f^{(H)}$. Eliminating t,

$$y \;=\; r_f^{(H)} + \varrho(\mathbf{w}^\star)\,(x + r_f^{(H)}), \tag{9}$$

a straight line in the (ES, expected return) plane with slope equal to the optimal ratio, through the riskless point $(-r_f^{(H)}, r_f^{(H)})$ and through P, where it touches the frontier. It is the analogue of the capital market line. Beyond P it would require borrowing at the risk-free rate; it is drawn for reference only, since the optimisation is fully invested without leverage.

7 Random portfolios

To picture the admissible region the notebook evaluates $N = 3\,000$ random portfolios. A vector $\mathbf{d}$ is drawn from a Dirichlet distribution with concentration 0.5 on every coordinate (non-negative entries that sum to one; a concentration below one favours corners, so the cloud reaches the single assets) and shifted onto the lower bounds, $\mathbf{w} = \boldsymbol{\ell} + (1 - \mathbf{1}^\top\boldsymbol{\ell})\,\mathbf{d}$, which satisfies the budget and the lower bounds by construction. Draws that break an upper bound are rejected and the batch is repeated until N portfolios survive. If fewer than 5% of the first batch survive, the bounds leave a very small admissible region: sampling stops after that batch, a message reports how many points were kept, and the cloud stays small. The cloud is illustrative only and never enters the optimisation. Each portfolio's expected return and ES are computed on all S scenarios, in chunks of 500 portfolios to keep memory bounded, with the same ES function as everything else.

8 Outputs and the published run

8.1 The two charts

Figure 1 is the main chart. In the plane (ES at the horizon, expected return at the horizon) it shows the random cloud, the mean–ES frontier, the four single assets (as reference points, even when a 100% position lies outside the bounds), the minimum-ES and maximum-ratio portfolios, the riskless point and the allocation line (9).

Figure 2 shows the simulated distribution at the horizon of each asset and of the two optimal portfolios, with a normal density of the same mean and standard deviation drawn on top so that the skewness and the fat tails inherited from the bootstrap are visible. Each panel spans ± 4 standard deviations of its own series. A table gives, for the same six series: mean, median, standard deviation, $\mathrm{VaR}_{95\%}$ (a quantile of returns), $\mathrm{ES}_{95\%}$ (a positive loss), probability of a loss, worst and best scenario.

8.2 Results of the run of 2026-10-29

Prices from 2010-12-15 to 2026-10-27 (4 027 daily returns), horizon 60 days, 10 000 scenarios, $\alpha = 0.95$, $r_f = 0$, bounds $(0, 1)$ on every asset.

The minimum-ES portfolio is almost entirely in bonds, split between government and investment-grade credit, with a 60-day Expected Shortfall of about 4%: with the 2020 and 2022 episodes in the sample, even euro bonds are far from riskless over three months. The maximum-ratio portfolio takes about 39% equity, 54% government bonds and very little credit: at this horizon, and with the tail dependence of the sample, high-yield credit adds tail risk without enough expected return to pay for it relative to the equity–government mix.

Two properties of the results are worth knowing before interpreting the weights.

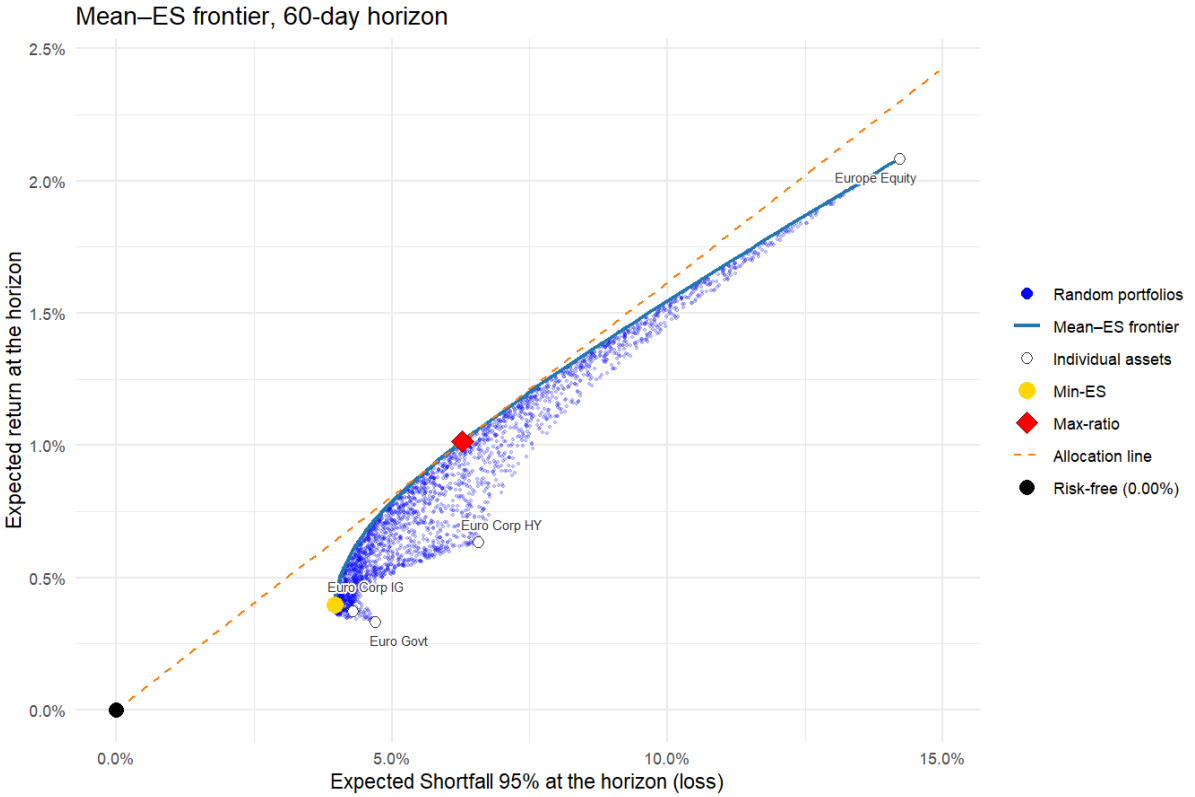


Figure 1: Mean–ES frontier, random portfolios, single assets, the two optimal portfolios and the allocation line (run of 2026-10-29).

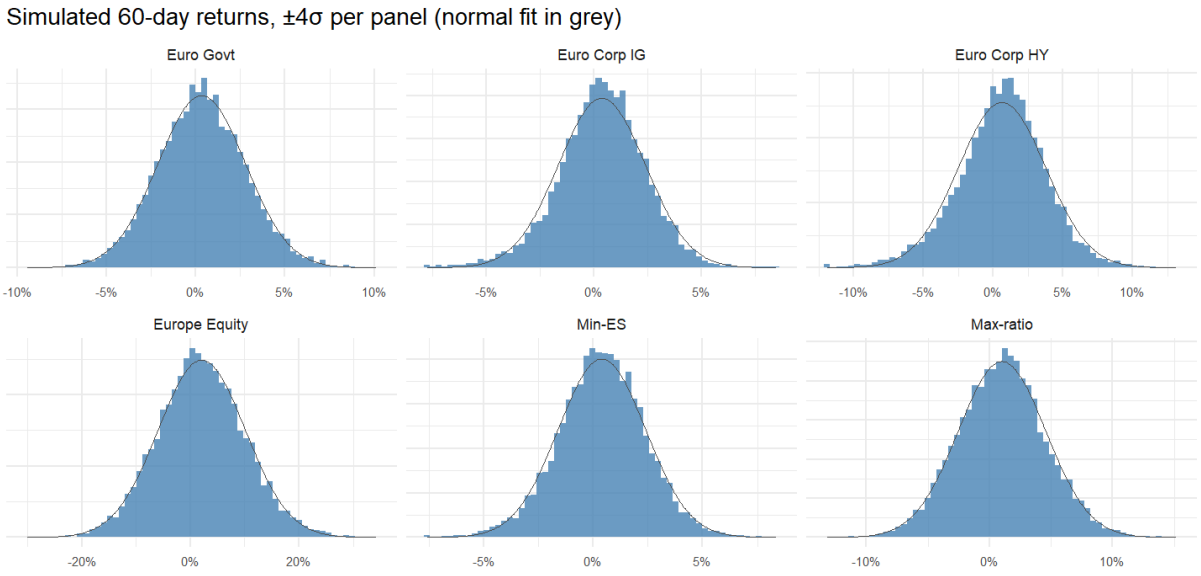


Figure 2: Simulated 60-day returns of the four assets and of the two optimal portfolios, with a normal fit in grey.

Portfolio	$\mathbb{E}[\rho]$	$\mathrm{ES}_{95\%}$	ratio
Minimum-ES	0.40%	3.96%	0.101
Max-ratio	1.02%	6.28%	0.162

Weight (%)	Min-ES	Max-ratio
Euro Govt	43.25	53.63
Euro Corp IG	46.36	7.59
Euro Corp HY	9.33	0.00
Europe Equity	1.06	38.79

Table 2: Optimal portfolios in the published run (returns and ES over the 60-day horizon).

- **The ratio is flat near its optimum.** On the same scenarios a mix of 40% government, 23% investment grade and 37% equity reaches a ratio of 0.161 against 0.162 at the optimum. A different bootstrap draw can move the maximum-ratio weights a lot while the ratio barely changes; the minimum-ES weights are stable.
- **Two runs differ by Monte Carlo error only.** On prices downloaded the same day (one more trading day, 4 028 returns), an independent implementation of the same method that draws its own scenarios gives $\mathrm{ES}_{95\%} = 4.02\%$ for the minimum-ES portfolio and a maximum ratio of 0.165. Fed with *identical* scenarios, the two optimisers agree to about 10^{-12} (Section 13).

9 Limitations and possible extensions

- **Independent days.** The i.i.d. bootstrap ignores volatility clustering and autocorrelation. Resampling whole weeks (`RETURN_FREQ = "weekly"`) keeps the dependence inside each week, with a sample five times smaller; a block or stationary bootstrap would keep longer dependence, at the price of choosing a block length.
- **Historical drift.** Expected returns are sample means over 2010–2026. Over 60 days they are small relative to the dispersion, but they set the ranking of the assets, which the ratio and the frontier respond to. Nothing in the project forecasts returns.
- **Buy-and-hold.** No rebalancing within the horizon and no transaction costs.
- **ETF proxies.** Fund costs and tracking differences are embedded in the series, and the common history is limited by the youngest fund.
- **Single period, single horizon.** Multi-period or rolling-window analysis is out of scope, as is any estimate of the sampling error of the optimal weights (a bootstrap of the bootstrap would give one).

Part II
The code

10 Repository layout

The repository is published on GitHub as `portfolio-optimization-r` under the MIT licence. Every tracked file is listed below, together with this document in `docs/`; the only generated folder, `data/cache/`, is excluded by `.gitignore` and is created at the first run.

```
portfolio-optimization-r/
|-- Asset Allocation EUR.Rmd        the notebook (source)
|-- Asset Allocation EUR.html       the notebook as last rendered
|-- README.md                       overview, how to run, example output
|-- LICENSE                         MIT
|-- install.R                       installs the required packages
|-- portfolio-optimization-r.Rproj  RStudio project file
```

```
|-- .gitignore                       R session files, caches, data/cache/
|-- R/                               the code, one file per stage
|   |-- all.R                        sources the other files in order
|   |-- params.R                     parameters and check_bounds()
|   |-- data.R                       download, cache, prices, log returns
|   |-- simulation.R                 bootstrap of horizon returns
|   |-- metrics.R                    portfolio return, ES, ratio
|   |-- optimisation.R               the linear programs (HiGHS)
|   |-- cloud.R                      random portfolios
|   |-- stats.R                      tables and the allocation line
|   `-- plots.R                      the two charts (ggplot2)
|-- tests/
|   |-- testthat.R                   entry point: runs tests/testthat/
|   |-- testthat/                    unit tests, one file per module
|   |   |-- helper-fixtures.R        sources R/all.R, synthetic data, helpers
|   |   |-- test-params.R            test-data.R, test-frequency.R,
|   |   |-- ...                      test-simulation.R, test-metrics.R,
|   |   `-- test-cross-check.R       test-optimisation.R, test-frontier.R,
|   |                                test-cloud.R, test-output.R
|   |-- cross_check/                 fixtures from the reference implementation
|   |   |-- reference_scenarios.csv  10,000 x 4 scenarios
|   |   `-- reference_expected.json  its optima and frontier on those scenarios
|   `-- verify_real_data.R           brute-force checks on the real data
|-- figures/
|   |-- frontier.png                 Figure 1, written by the notebook
|   `-- distributions.png            Figure 2, written by the notebook
|-- docs/
|   |-- project-guide.tex / .pdf     this document
|   `-- specs/                       design notes of later changes
`-- data/cache/                      (generated, not tracked) price cache
```

The design rule is one file per stage of the pipeline, in the order in which the notebook uses them, and no package structure: the notebook sources `R/all.R`, which sources the eight files, and every function is then available in the global environment. This keeps the project runnable by opening the `.Rmd` and clicking *Knit*, with nothing to build.

10.1 Packages

`install.R` installs everything needed with one `install.packages()` call: `quantmod` (download from Yahoo Finance; it brings `xts` and `zoo` for time series), `highs` (the HiGHS solver) and `Matrix` (sparse matrices for the LP), `ggplot2` and `ggrepel` (charts; `scales`, used for the percent axes, comes with `ggplot2`), `jsonlite` (cache metadata), `rmarkdown` and `knitr` (rendering), `testthat` (tests). In the `R/` files and in the notebook every call into a package is written with its prefix (`highs::highs_solve`, `ggplot2::ggplot`, ...), so the reader always sees where a function comes from and no `library()` call is needed.

11 The notebook: `Asset Allocation EUR.Rmd`

The notebook is an R Markdown document rendered to HTML with a table of contents. Its prose restates the method of Part I in a few paragraphs per stage, and four chunks contain code (plus a hidden `setup` chunk that sets the chunk options and loads `quantmod` quietly).

`source`. One line, `source("R/all.R")`, which assumes the working directory is the repository root. RStudio's *Knit* and `rmarkdown::render()` both run the document from its own folder, and the `.Rmd` sits in the repository root, so this always holds.

parameters. Restates `BOUNDS`, the per-asset minimum and maximum weights, so that a reader can edit them without leaving the notebook, and prints every other parameter as defined in `R/params.R`. An asset not listed in `BOUNDS` gets $(0, 1)$; the bounds are validated when the run starts.

run. The pipeline, in the order of Part I. Read it as the table of contents of the `R/` folder:

```
data <- get_prices_and_logret(TICKERS)                        # R/data.R
logret <- data$logret; colnames(logret) <- NAMES[...]   # asset-class labels
logret  <- aggregate_logret(logret, RETURN_FREQ)              # daily or weekly
horizon <- horizon_periods(HORIZON_DAYS, RETURN_FREQ)         # R/params.R
h_label <- horizon_label(HORIZON_DAYS, RETURN_FREQ)
b <- check_bounds(BOUNDS, colnames(logret)); lo <- b$lo; hi <- b$hi
rf_h <- exp(RISK_FREE * HORIZON_DAYS / 252) - 1               # annual -> horizon

scenarios <- simulate_horizon_returns(logret, horizon,        # R/simulation.R
                                      N_SCENARIOS, SIM_SEED)
asset_ret <- colMeans(scenarios)
asset_es  <- apply(scenarios, 2, expected_shortfall, level = ES_LEVEL)

rand      <- sample_random_portfolios(scenarios, ES_LEVEL, lo, hi,       # R/cloud.R
                                      N_RANDOM, DIRICHLET_ALPHA, CLOUD_SEED)
min_es    <- min_es_portfolio(scenarios, ES_LEVEL, lo, hi, rf_h)  # R/optimisation.R
max_ratio <- max_ratio_portfolio(scenarios, ES_LEVEL, lo, hi, rf_h)
frontier  <- build_es_frontier(scenarios, ES_LEVEL, lo, hi, rf_h,
                               N_FRONTIER, min_es$ret,
                               0.999 * max_return(scenarios, lo, hi))

p_frontier <- plot_frontier(...)                              # R/plots.R
cat(...); knitr::kable(weights_table(min_es, max_ratio))      # R/stats.R
```

The chunk prints one `[data]` line saying where the prices came from, the sample summary, the two optimal portfolios and their weights, and saves Figure 1 to `figures/frontier.png`.

distributions. Binds the two optimal portfolios' scenario returns to the asset scenarios as two extra columns, computes the statistics table with `horizon_stats()`, formats it in percent and draws Figure 2, saved to `figures/distributions.png`.

12 The `R/` folder, file by file

Each file starts with a comment stating its role, and each function with a docstring-style comment stating what it returns and what is not obvious about it. The non-trivial blocks inside the functions carry a comment explaining the step. This section gives the map; the comments give the details.

12.1 `params.R`: parameters, horizon helpers and `check_bounds()`

All tunable quantities live here, as plain global constants (Table 3). `NAMES` maps each Yahoo ticker to the asset-class label used in every table and chart; `TICKERS` is its set of names. The notebook renames the columns of the log-return matrix from tickers to labels right after the download, so from the simulation on everything is labelled by asset class.

Three functions follow the constants. `horizon_periods(horizon_days, freq)` returns the number of periods the bootstrap must draw: the days themselves, or `horizon_days / 5` for weekly returns, stopping if that is not a whole number. `horizon_label()` builds the text of the chart titles from the same two arguments (`"60-day"` or `"12-week (60 business days)"`).

Constant	Meaning	Value
START_DATE	first date requested from Yahoo	2010-01-01
HORIZON_DAYS	horizon H in business days	60
RETURN_FREQ	returns resampled: "daily" or "weekly"	"daily"
N_SCENARIOS	bootstrap scenarios S	10 000
ES_LEVEL	confidence level α	0.95
N_RANDOM	random portfolios in the cloud	3 000
DIRICHLET_ALPHA	concentration of the random weights	0.5
N_FRONTIER	target returns along the frontier	50
RISK_FREE	annual risk-free rate r_f	0
SIM_SEED, CLOUD_SEED	seeds of the two generators	42, 7
BOUNDS	per-asset (ℓ_i, u_i)	$(0, 1)$ each
CACHE_DIR	folder of the price cache	data/cache
CACHE_TTL_HOURS	age after which the cache is refreshed	12

Table 3: Parameters in R/params.R and their default values.

check_bounds(bounds, names) turns the named list BOUNDS into two numeric vectors lo and hi ordered like names, filling $(0, 1)$ for assets not listed, and stops with a message naming the offending asset when an entry is not a pair of numbers, when $\ell_i < 0$, $u_i > 1$ or $\ell_i > u_i$, when BOUNDS names an asset that is not in the universe (a typo would otherwise be silently ignored), or when $\sum \ell_i > 1$ or $\sum u_i < 1$, in which case no fully invested portfolio exists. The two sum checks allow a tolerance of 10^{-9} against floating-point rounding in the sums.

12.2 data.R: download, cache, prices and log returns

Five functions, used in this order.

yahoo_download(tickers, start) downloads each ticker with quantmod::getSymbols(), keeps the *Adjusted* column and returns a plain numeric matrix with one row per date and one column per requested ticker. Three points are not obvious. The call for each ticker sits inside tryCatch, so a ticker that fails (network error, unknown symbol, no *Adjusted* column) gives a column of NA instead of aborting the whole download; the cache layer treats such a column as a failed ticker. The series are merged on the union of their dates, and rows where every asset is NA are dropped: quantmod returns such rows on exchange holidays, and left in place they would later be forward-filled into fake zero-return days. Finally the result is rebuilt as a matrix with exactly the requested columns, so that a missing ticker is visible as an all-NA column rather than as a missing one.

.cache_key(tickers, start) is the first 12 hexadecimal characters of the MD5 of the sorted tickers and the start date, so that a different universe or a different start date gets its own cache file and the order of the tickers does not matter.

load_close() takes tickers, start, ttl_hours, force_refresh, cache_dir and downloader. It implements the lookup order *fresh cache* → *download* → *stale cache* → *error* in four numbered blocks. (1) If a cache file and its JSON metadata exist, their age is computed from the fetched_at timestamp (UTC); if it is below the time-to-live and no refresh is forced, the cached matrix is returned with meta$source set to "cache (2.3h old)". (2) Otherwise the downloader is called, and the tickers that came back empty or missing are listed in failed. (3a) If the download threw an error: return the stale cache if there is one, else stop. (3b) If some ticker failed: return the stale cache if there is one, else return the partial data

without writing it, so that a good cache is never overwritten. (3c) If every ticker is present: write the RDS file and the metadata and return the fresh data. The `downloader` argument exists so that the tests can inject a fake download function and exercise every branch without touching the network.

`get_prices_and_logret()` takes `tickers`, `start` and `force_refresh` (further arguments go to `load_close()`); it calls `load_close()`, prints the `[data] <source>` line, sorts the rows chronologically, drops tickers with no data at all, forward-fills missing prices column by column with an explicit loop ("a missing price is replaced by the last available one"), keeps only the dates on which every asset has a price, and returns `prices` and `logret = diff(log(prices))`, both matrices with date row names.

`aggregate_logret(logret, freq)` returns its input for `"daily"`. For `"weekly"` it labels every row with its ISO year-week (`format(date, "%G-%V")`), drops the rows of the first and last week, and sums the daily log returns week by week with a loop over the weeks, dating each result at the week's last trading day. The notebook calls it right after the download, so everything downstream sees one matrix of returns and never asks which frequency it has.

12.3 `simulation.R`: the bootstrap

One function, `simulate_horizon_returns(logret, horizon, n_scenarios, seed)`, implementing (3). It seeds the generator, draws *one* vector of $S \times H$ day indices with `sample.int()`, and then, asset by asset, picks the daily log returns on those days, arranges them as an $S \times H$ matrix and sums each row. Because the same index vector is reused for every asset, each scenario is a sum of whole historical days, which is the point of the method. The result is $\exp(\cdot) - 1$ of the summed log returns, with the asset labels as column names. A comment in the file explains how `matrix()` fills its cells column by column and why that is harmless here: which draws end up in which scenario does not matter, since all draws are independent.

12.4 `metrics.R`: portfolio return, ES and ratio

Four short functions that define the vocabulary of the rest of the code. `portfolio_scenarios(R, w)` is (1), a matrix–vector product. `expected_shortfall(returns, level)` is (4) written out in steps: losses, tail size m, whole part k, sorted losses, sum of the k largest, fractional share of the next one, average. It is the single definition of ES in the project; the cloud and the statistics table call it, and the tests check it against the LP. `expected_return(R, w)` is $\bar{\mathbf{r}}^\top \mathbf{w}$ and `excess_ratio(R, w, level, rf_h)` is (5), computed on excess returns in both numerator and denominator.

12.5 `optimisation.R`: the linear programs

The file opens with a comment explaining what a linear program is and in which form `highs::highs_solve()` expects it, through the arguments `L`, `lower`, `upper`, `A`, `lhs` and `rhs`: minimise $L^\top x$ subject to $lhs \le Ax \le rhs$ and $lower \le x \le upper$. Every optimisation in the project is expressed in this form.

`solve_es_lp()` takes `R`, `level`, the weight bounds `w_lower` and `w_upper`, the optional rows `extra_eq` and `extra_ub` and the flag `budget`; it builds and solves (7). The variable vector is $x = [\mathbf{w} \mid \zeta \mid \mathbf{u}]$ and the function assembles its pieces in labelled blocks: the objective (zeros for $\mathbf{w}$, one for ζ, $1/((1-\alpha)S)$ for each u_s); the S scenario rows as the sparse matrix $[-R \mid -\mathbf{1} \mid -I_S]$ with right-hand side 0; optional inequality rows `extra_ub` and equality rows `extra_eq` on the weights only, padded with zero columns for ζ and $\mathbf{u}$; the budget row $\mathbf{1}^\top \mathbf{w} = 1$ when `budget =`

TRUE; and the variable bounds ($\mathbf{w}$ within its bounds, ζ free, $\mathbf{u} \geq 0$). It stops with the solver's status message unless that is "Optimal", and returns the first n entries of the solution (the weights) together with the objective value (the ES). The function is generic on purpose: the three portfolios below differ only in the arguments they pass.

portfolio_result(R, w, level, rf_h) packages a weight vector with its expected return, ES and ratio, all recomputed directly on w with the functions of metrics.R rather than read from the solver, so that every portfolio is described in exactly the same way.

min_es_portfolio(R, level, lo, hi, rf_h, target_return) calls solve_es_lp() with the bounds and, when a target is given, the single inequality row $-\bar{\mathbf{r}}^\top \mathbf{w} \leq -r_{\text{target}}$ (the solver takes "$\leq$" rows, so "$\geq$" is written with flipped signs). Without a target it is the minimum-ES portfolio; with one it is a frontier point.

max_return(R, lo, hi) is the tiny LP for the top of the frontier, written directly against highs_solve(): minimise $-\bar{\mathbf{r}}^\top \mathbf{w}$ subject to the budget and the bounds, and return the negated optimum.

max_ratio_portfolio(R, level, lo, hi, rf_h) implements Section 6.3. It first checks precondition 1 through max_return(). It then forms the excess returns R^e and their means, builds the $2n$ homogeneous bound rows (8) with an explicit loop over the assets (each row is ℓ_i on every column and $\ell_i - 1$ on column i, or $-u_i$ everywhere and $1 - u_i$ on column i), and calls solve_es_lp() on R^e with variable bounds $[0, \infty)$, the equality $\bar{\mathbf{r}}^{e\top} \mathbf{y} = 1$, those $2n$ rows and budget = FALSE. After the solve it checks precondition 2 (the optimal ES must be positive), normalises $\mathbf{w} = \mathbf{y} / \mathbf{1}^\top \mathbf{y}$ and returns a portfolio_result(). The comment above the function gives the two-step derivation of Section 6.3 in words.

build_es_frontier() takes R, level, lo, hi, rf_h, n_points, r_lo and r_hi; it loops over n_points equally spaced targets between r_lo and r_hi, calls min_es_portfolio() for each inside tryCatch, skips a target that fails with a [frontier] message, and returns the ES and expected return of every point plus the matrix of their weights.

12.6 cloud.R: random portfolios

es_rows(P, level) applies expected_shortfall() to every row of a portfolios×scenarios matrix with a loop; it exists so that the cloud uses the same ES definition as the optimiser. sample_random_portfolios(R, level, lo, hi, n, alpha, seed) implements Section 7 in two labelled blocks. The first draws batches of n Dirichlet vectors (gamma draws divided by their row sums), shifts them onto the lower bounds, keeps the rows that respect every upper bound (with a 10^{-12} slack so that a weight exactly equal to its bound passes), and repeats until n rows have been kept, with the early stop after a first batch that keeps fewer than 5% of its rows. The second evaluates the kept portfolios on all scenarios in chunks of 500 rows, computing $P = WR^\top$ for each chunk, its row means (expected returns) and its row-wise ES. The result is a data frame with one row per portfolio and columns ret and es; it may have fewer than n rows, or none, and the plotting code copes with both.

12.7 stats.R: tables and the allocation line

weights_table(min_es, max_ratio) is the two-column table of weights in percent printed by the notebook. allocation_line(max_ratio, rf_h, xs) evaluates (9) at the abscissae xs; its comment re-derives the line from the mixing argument. horizon_stats(returns, level)

computes, for every column of a returns matrix, the eight statistics of the distribution table (mean, median, standard deviation, VaR as the $(1-\alpha)$ quantile of returns, ES through `expected_shortfall()`, probability of a loss, minimum and maximum) and names the two tail columns after the level (`var_95`, `es_95`), so that a change of `ES_LEVEL` is visible in the table header.

12.8 `plots.R`: the two charts

Both functions return a `ggplot` object; the notebook prints it and saves it with `ggsave()`.

`plot_frontier()` takes the cloud `rand`, the frontier (`frontier_es`, `frontier_ret`), the two portfolios `min_es` and `max_ratio`, the single assets (`asset_ret`, `asset_es`), `rf_h`, the text `horizon_label` and `level`. It builds one small data frame per series (cloud, frontier sorted by ES so that it is drawn left to right, single assets with their labels, the two optimal portfolios, the allocation line from $-r_f^{(H)}$ to a little beyond the riskiest point on the chart, the riskless point) and stacks the layers from background to foreground: cloud, frontier, dashed line, white-filled asset markers, the gold minimum-ES point, the red diamond of the maximum-ratio portfolio, the black riskless point and finally the labels, placed by `ggrepel` with a white halo and pushed away from the three marked points as well. The legend follows a fixed order and its keys are enlarged so that they stay readable; the x-axis title is built from `level` and the chart title from `horizon_label`. The three non-ASCII characters in the titles (en dash, plus–minus, sigma) are written as `\u` escapes so that the file is plain ASCII and renders the same in every locale.

`plot_distributions()` takes `returns`, `horizon_label` and `n_sigma` (default 4) and draws, for every column of `returns`, a density histogram of the observations clipped to mean $\pm$ `n_sigma` standard deviations and the normal density with the same mean and standard deviation, in a grid of panels with free axes and in the column order of the input (a factor with explicit levels, otherwise `ggplot2` would sort the panels alphabetically).

12.9 `all.R`

Eight `source()` calls in dependency order: `params.R`, `data.R`, `simulation.R`, `metrics.R`, `optimisation.R`, `cloud.R`, `stats.R` and `plots.R`. The notebook, the tests and `verify_real_data.R` all start by sourcing this file from the repository root.

13 Tests and verification

Correctness is checked at three levels, all in the repository.

13.1 Unit tests on synthetic data: `tests/testthat/`

`tests/testthat.R` is the conventional entry point (`library(testthat)` followed by `test_dir("tests/testthat")`). `helper-fixtures.R` is loaded automatically before the test files: it sources `R/all.R` from the repository root and defines the fixtures shared by the tests. `synthetic_logret()` returns 500 days of normal daily log returns for four assets with fixed means, volatilities and seed; `sim()` bootstraps 2 000 scenarios from a log-return matrix; `simplex_grid(n_assets, step)` enumerates every weight vector on a grid of the given step that sums to one; `apply_bounds(W, lo, hi)` keeps the rows of a weight matrix that respect the bounds.

There is one test file per module, 52 test blocks and 175 expectations in all:

- `test-params.R`: the parameters exist; `check_bounds()` fills defaults, keeps the asset order, rejects negative, excessive, inverted and malformed bounds and unknown asset names, and accepts sums that equal one up to floating-point rounding.
- `test-data.R`: with a fake downloader and a temporary cache folder, `load_close()` downloads, writes the cache and then serves it; falls back to the stale cache when the download fails; errors when it fails with no cache; never overwrites the cache with a partial download; returns a partial download without writing it. `get_prices_and_logret()` returns log returns, drops empty tickers, forward-fills and errors on an empty result; `yahoo_download()` drops all-`NA` holiday rows.
- `test-frequency.R`: `aggregate_logret()` is the identity for daily returns; on a calendar-day series every weekly return equals the log ratio of the week-end prices rebuilt from the daily ones, is dated at the week's last day and consecutive rows are seven days apart; a hand-built case covers an ISO week split across two years and a week with one trading day; the first and last weeks are dropped; `horizon_periods()` maps 60 days to 12 weeks and rejects 61; unknown frequencies error.
- `test-simulation.R`: shape and column names; with a horizon of one day every scenario is a historical day; the same seed gives the same matrix; two perfectly correlated assets stay perfectly correlated after the bootstrap (the cross-section is preserved).
- `test-metrics.R`: `expected_shortfall()` against hand-computed examples and against the plain tail mean; invalid levels error; the portfolio helpers are mutually consistent.
- `test-optimisation.R`: the LP optimum equals the ES recomputed on the optimal weights to 10^{-8}, with $S = 2\,000$ (integer tail) and $S = 1\,001$ (fractional tail); no point of a 5% grid over the simplex has a lower ES than the minimum-ES portfolio, and none, with or without bounds and with $r_f^{(H)} = 0.002$, has a higher ratio than the maximum-ratio portfolio; a target return is honoured and costs ES; the bounds are honoured; a riskless asset is chosen fully; the maximum-ratio preconditions raise on data built to violate them; `max_return()` respects the bounds.
- `test-frontier.R`: the frontier is monotone, starts at the minimum-ES portfolio and has the requested number of points; the maximum-ratio portfolio lies on it; infeasible targets are skipped with a message.
- `test-cloud.R`: shape; with bounds, every point is kept and none has a higher expected return than the bounded maximum; tight bounds give fewer rows and a message; `es_rows()` matches `expected_shortfall()` row by row with a fractional tail.
- `test-output.R`: `horizon_stats()` matches a hand example; the allocation line passes through the maximum-ratio point and the riskless point with $r_f^{(H)} = 0.002$; the weights table has the right shape; both charts build and render on a headless device; `plot_frontier()` copes with an empty cloud.
- `test-cross-check.R`: see below.

The suite takes about a minute and a half, most of it in the cross-check.

13.2 Cross-check against a reference implementation: `tests/cross_check/`

`reference_scenarios.csv` holds a $10\,000 \times 4$ scenario matrix produced by an independent implementation of the same method on prices downloaded the same day, and `reference_expected.json` the minimum-ES portfolio, the maximum-ratio portfolio and the 50-point frontier that this implementation computed on those scenarios, both with the trivial bounds and with the tight bounds {Equity $\leq 20\%$, Govt $\geq 10\%$}. `test-cross-check.R` feeds the same scenarios to the R optimiser and compares weights (10^{-5}), expected returns, ES, ratios and all frontier points (10^{-6}); the observed agreement is about 10^{-12}. The test therefore isolates the optimisation from the simulation: any difference between two full runs of the two implementations is Monte Carlo error from different scenario draws, not a difference in what is being optimised.

13.3 Brute-force checks on the real data: `tests/verify_real_data.R`

A script rather than a test, run from the repository root with `Rscript tests/verify_real_data.R`. It uses the real prices (from the cache if fresh, otherwise downloaded) and the published seed and exits with status 1 if any check fails. For the trivial bounds and for the tight bounds it checks: (1) the LP objective and the directly computed ES of the optimal weights agree to 10^{-8}, for both portfolios; (2) a 2% grid over the bounded simplex (about 23 000 portfolios) never beats the LP in ES or in ratio, and all optimal and frontier weights lie within the bounds and sum to one; (3) the frontier ES is non-decreasing and the maximum-ratio portfolio lies on the frontier; (4) a second run with the same seed reproduces the scenarios and the minimum-ES weights exactly. It takes about two minutes and prints one `PASS` or `FAIL` line per check.

14 Running and modifying the project

14.1 First run

Requires R $\geq$ 4.3 and RStudio (for *Knit*; from a plain R session, `rmarkdown::render()` needs `pandoc` on the path or the `RSTUDIO_PANDOC` environment variable pointing at the folder that contains it). Then:

1. open `portfolio-optimization-r.Rproj` in RStudio;
2. run `source("install.R")` once;
3. open `Asset Allocation EUR.Rmd` and click *Knit*, or run `rmarkdown::render("Asset Allocation EUR.Rmd")`.

The first run downloads the prices (a few seconds) and creates `data/cache/`; later runs within 12 hours use the cache. Rendering takes about a minute, most of it in the 50 frontier LPs. The HTML and the two PNG files in `figures/` are overwritten at every run.

14.2 Common changes

- **Constrain an asset class.** Edit `BOUNDS` in the `parameters` chunk of the notebook, for example `"Europe Equity" = c(0, 0.2)` for at most 20% equity, and re-knit. The frontier, the cloud and both optimal portfolios respect the new bounds; single assets are still plotted as reference points.
- **Change the horizon, the number of scenarios or the ES level.** Edit the constant in `R/params.R`. The level appears in the axis title and in the column names of the statistics table (`var_97.5`, `es_97.5` for 0.975). With a fractional tail size ($(1-\alpha)S$ not an integer) the ES formula and the LP still agree exactly.
- **Resample weeks instead of days.** Set `RETURN_FREQ` to `"weekly"` in `R/params.R`. The sample line of the notebook then reports weekly returns and the charts a `12-week (60 business days)` horizon; `HORIZON_DAYS` must be a multiple of 5. Expect the optimal weights to move, since the resampled distribution has about five times fewer support points, keeps the within-week dependence and covers slightly fewer trading days (Section 3).
- **Use a non-zero risk-free rate.** Set `RISK_FREE` to the annual rate; the notebook converts it to the horizon. The riskless point then moves to $(-r_f^{(H)}, r_f^{(H)})$ and the ratio is computed on excess returns.
- **Change the universe.** Edit `NAMES` in `R/params.R` (ticker $\rightarrow$ label). Every function takes the number of assets from the data; the only other edit is `BOUNDS`, in the notebook's `parameters` chunk and in `R/params.R` for the tests, whose names must be labels of the new universe, otherwise `check_bounds()` stops. A new universe gets its own cache file. The common sample starts at the youngest listing, so adding a recently launched fund shortens the history for all assets.

- **Force a fresh download.** Call `get_prices_and_logret(TICKERS, force_refresh = TRUE)` or delete `data/cache/`.

14.3 Running the tests

From the repository root, in R: `testthat::test_dir("tests/testthat")`; from a shell: `Rscript tests/verify_real_data.R`. The testthat suite never touches the network: `test-data.R` injects a fake downloader and mocks `quantmod::getSymbols()`, and the other files work on synthetic data or on the cross-check fixtures. `verify_real_data.R` may download prices.

References

[1] R. T. Rockafellar and S. Uryasev, *Optimization of conditional value-at-risk*, Journal of Risk 2(3), 21–41, 2000.

[2] R. T. Rockafellar and S. Uryasev, *Conditional value-at-risk for general loss distributions*, Journal of Banking & Finance 26(7), 1443–1471, 2002.

[3] P. Artzner, F. Delbaen, J.-M. Eber and D. Heath, *Coherent measures of risk*, Mathematical Finance 9(3), 203–228, 1999.

[4] A. Charnes and W. W. Cooper, *Programming with linear fractional functionals*, Naval Research Logistics Quarterly 9, 181–186, 1962.

[5] B. Efron, *Bootstrap methods: another look at the jackknife*, Annals of Statistics 7(1), 1–26, 1979.

[6] Q. Huangfu and J. A. J. Hall, *Parallelizing the dual revised simplex method*, Mathematical Programming Computation 10(1), 119–142, 2018.